Activity 1.7

1 Using software on your computer (for example, text colour option in *Word*), find out what colours would be represented by the following RGB denary value combinations:

a	Red	53	**b**	Red	201	**c**	Red	12
	Green	55		Green	122		Green	111
	Blue	139		Blue	204		Blue	81

2 Convert each of the above denary numbers into hexadecimal.

1.1.4 Addition of binary numbers

This section will look at the addition of two 8-bit positive binary numbers.

Note the following key facts when carrying out ***addition*** of two binary digits:

binary addition	carry	sum
0+0	0	0
0+1	0	1
1+0	0	1
1+1	1	0

This can then be extended to consider the addition of three binary digits:

binary digit	carry	sum
0+0+0	0	0
0+0+1	0	1
0+1+0	0	1
0+1+1	1	0
1+0+0	0	1
1+0+1	1	0
1+1+0	1	0
1+1+1	1	1

For comparison: if we add 7 and 9 in denary the result is: carry = 1 and sum = 6; if we add 7, 9 and 8 the result is: carry = 2 and sum = 4, and so on.

Advice

Here's a quick recap on the role of carry and sum. If we want to add the numbers 97 and 64 in decimal, we:

- add the numbers in the right hand column first
- if the sum is greater than 9 then we carry a value to the next column
- we continue moving left, adding any carry values to each column until we are finished.

For instance:

```
  9 7
+ 6 4
  1 1  CARRY VALUES
-------
1 6 1  SUM VALUES
```

Adding in binary follows the same rules except that we carry whenever the sum is greater than 1.

Example 1

Add 00100111 + 01001010

We will set this out showing carry and sum values:

```
  0 0 1 0 0 1 1 1
+ 0 1 0 0 1 0 1 0
  -----------------
    1 1 1          <-- carry values
  -----------------
  0 1 1 1 0 0 0 1  <-- sum values
```

Answer: **01110001**

column 1: 1 + 0 = 1 no carry
column 2: 1 + 1 = 0 carry 1
column 3: 1 + 0 + 1 = 0 carry 1
column 4: 0 + 1 + 1 = 0 carry 1
column 5: 0 + 0 + 1 = 1 no carry
column 6: 1 + 0 = 1 no carry
column 7: 0 + 1 = 1 no carry
column 8: 0 + 0 = 0 no carry

Example 2

a Convert 126 and 62 into binary.
b Add the two binary values in part a and check the result matches the addition of the two denary numbers

a 126 = 0 1 1 1 1 1 1 0 and 62 = 0 0 1 1 1 1 1 0

b

```
  0 1 1 1 1 1 1 0
+ 0 0 1 1 1 1 1 0
  -----------------
  1 1 1 1 1 1      <-- carry values
  -----------------
  1 0 1 1 1 1 0 0  <-- sum values
```

Answer: **10111100**

column 1: 0 + 0 = 0 no carry
column 2: 1 + 1 = 0 carry 1
column 3: 1 + 1 + 1 = 1 carry 1
column 4: 1 + 1 + 1 = 1 carry 1
column 5: 1 + 1 + 1 = 1 carry 1
column 6: 1 + 1 + 1 = 1 carry 1
column 7: 1 + 0 + 1 = 0 carry 1
column 8: 0 + 0 + 1 = 1 no carry

1 0 1 1 1 1 0 0 has the equivalent denary value of 128 + 32 + 16 + 8 + 4 = 188 which is the same as 126 + 62.

Activity 1.8

Carry out the following binary additions:

a 0 0 0 1 1 1 0 1 + 0 1 1 0 0 1 1 0
b 0 0 1 0 0 1 1 1 + 0 0 1 1 1 1 1 1
c 0 0 1 0 1 1 1 0 + 0 1 0 0 1 1 0 1
d 0 1 1 1 0 1 1 1 + 0 0 1 1 1 1 1 1
e 0 0 1 1 1 1 0 0 + 0 0 1 1 0 0 1 1
f 0 0 1 1 1 1 0 0 + 0 1 1 1 1 0 1 1
g 0 0 1 1 1 1 1 1 + 0 0 1 1 1 1 1 1
h 0 0 1 1 0 0 0 1 + 0 0 1 1 1 1 1 1
i 0 1 1 1 1 1 1 1 + 0 1 1 1 1 1 1 1
j 1 0 1 0 0 0 1 0 + 0 0 1 1 1 0 1 1

Activity 1.9

Convert the following denary numbers into binary and then carry out the binary addition of the two numbers and check your answer against the equivalent denary sum:

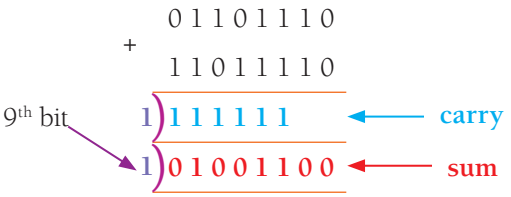
a 98 + 15
b 29 + 88
c 49 + 100
d 51 + 171
e 82 + 69
f 100 + 140
g 19 + 139
h 203 + 30
i 66 + 166
j 211 + 35

Overflow

Now consider the following example:

Example 3

Add 0 1 1 0 1 1 1 0 and 1 1 0 1 1 1 1 0 (using 8 bits)



This addition has generated a 9th bit. The 8 bits of the answer are 0 1 0 0 1 1 0 0 – this gives the denary value (64 + 8 + 4) of 76 which is incorrect because the denary value of the addition is 110 + 222 = 332.

The maximum denary value of an 8-bit binary number is 255 (which is $2^8 - 1$). The generation of a 9th bit is a clear indication that the sum has exceeded this value. This is known as an **overflow error** and in this case is an indication that a number is too big to be stored in the computer using 8 bits.

The greater the number of bits which can be used to represent a number then the larger the number that can be stored. For example, a 16-bit register would allow a maximum denary value of 65 535 (i.e. $2^{16} - 1$) to be stored, a 32-bit register would allow a maximum denary value of 4 294 967 295 (i.e. $2^{32} - 1$), and so on.

Activity 1.10

1 Convert the following pairs of denary numbers to 8-bit binary numbers and then add the binary numbers. Comment on your answers in each case:

   a 89 + 175 **b** 168 + 99 **c** 88 + 215

2 Carry out the following 16-bit binary additions and comment on your answers:

   a 0111 1111 1111 0001 + 0101 1111 0011 1001

   b 1110 1110 0000 1011 + 1111 1101 1101 1001

1.1.5 Logical binary shifts

Computers can carry out a **logical shift** on a sequence of binary numbers. The logical shift means moving the binary number to the ***left*** or to the ***right***. Each shift ***left*** is equivalent to ***multiplying*** the binary number by 2 and each shift ***right*** is equivalent to ***dividing*** the binary number by 2.

As bits are shifted, any empty positions are replaced with a zero – see examples below. There is clearly a limit to the number of shifts which can be carried out if the binary number is stored in an 8-bit register. Eventually after a number of shifts the register would only contain zeros. For example, if we shift 01110000 (denary value 112) five places left (the equivalent to multiplying by 2^5, i.e. 32), in an 8-bit register we would end up with 00000000. This makes it seem as though 112 × 32 = 0! This would result in the generation of an error message.

Example 1

The denary number 21 is 00010101 in binary. If we put this into an 8-bit register:

128	64	32	16	8	4	2	1
0	0	0	1	0	1	0	1

The left-most bit is often referred to as the MOST SIGNIFICANT BIT

If we now shift the bits in this register one place to the left, we obtain:

128	64	32	16	8	4	2	1
0	0	1	0	1	0	1	0

Note how the empty right-most bit position is now filled with a 0

The left-most bit is now lost following a left shift

The value of the binary bits is now 21×2^1 i.e. 42. We can see this is correct if we calculate the denary value of the new binary number 101010 (i.e. 32 + 8 + 2).

Suppose we now shift the original number two places left:

128	64	32	16	8	4	2	1
0	1	0	1	0	1	0	0

The binary number 1010100 is 84 in denary – this is 21×2^2.

And now suppose we shift the original number three places left:

1	0	1	0	1	0	0	0

The binary number 10101000 is 168 in denary – this is 21×2^3.

So, let us consider what happens if we shift the original binary number 00010101 four places left:

128	64	32	16	8	4	2	1
0	1	0	1	0	0	0	0

Losing 1 bit following a shift operation will cause an error

The left-most 1-bit has been lost. In our 8-bit register the result of 21×2^4 is 80 which is clearly incorrect. This error is because we have exceeded the maximum number of left shifts possible using this register.

Example 2

The denary number 200 is 11001000 in binary. Putting this into an 8-bit register gives:

128	64	32	16	8	4	2	1
1	1	0	0	1	0	0	0

The right-most bit is often referred to as the LEAST SIGNIFICANT BIT

If we now shift the bits in this register one place to the right:

128	64	32	16	8	4	2	1
0	1	1	0	0	1	0	0

Note how the left-most bit position is now filled with a 0

The value of the binary bits is now 200 ÷ 2^1 i.e. 100. We can see this is correct by converting the new binary number 01100100 to denary (64 + 32 + 4).

Suppose we now shift the original number two places to the right:

128	64	32	16	8	4	2	1
0	0	1	1	0	0	1	0

The binary number 00110010 is 50 in denary – this is 200 ÷ 2^2.

And suppose we now shift the original number three places to the right:

Notice the 1-bit from the right-most bit position is now lost causing an error

128	64	32	16	8	4	2	1
0	0	0	1	1	0	0	1

The binary number 00011001 is 25 in denary – this is 200 ÷ 2^3.

Now let us consider what happens if we shift four places right:

128	64	32	16	8	4	2	1
0	0	0	0	1	1	0	0

The right-most 1-bit has been lost. In our 8-bit register the result of 200 ÷ 2^4 is 12, which is clearly incorrect. This error is because we have therefore exceeded the maximum number of right shifts possible using this 8-bit register.

Example 3

a Write 24 as an 8-bit register.
b Show the result of a logical shift 2 places to the left.
c Show the result of a logical shift 3 places to the right.

a

128	64	32	16	8	4	2	1
0	0	0	1	1	0	0	0

b

128	64	32	16	8	4	2	1	
0	1	1	0	0	0	0	0	⇦ 24 × 2^2 = 96

c

128	64	32	16	8	4	2	1	
0	0	0	0	0	0	1	1	⇦ 24 ÷ 2^3 = 3

Example 4

a Convert 19 and 17 into binary.
b Carry out the binary addition of the two numbers.
c Shift your result from part **b** two places left and comment on the result.
d Shift your result from part **b** three places right and comment on the result.

a

128	64	32	16	8	4	2	1	
0	0	0	1	0	0	1	1	19
0	0	0	1	0	0	0	1	17

b

```
  0 0 0 1 0 0 1 1
+ 0 0 0 1 0 0 0 1
    1   1 1        ← carry
  0 0 1 0 0 1 0 0  ← sum
```

c

128	64	32	16	8	4	2	1
1	0	0	1	0	0	0	0

d

128	64	32	16	8	4	2	1
0	0	0	0	0	1	0	0

In **c** the result is $36 \times 2^2 = 144$ (which is correct).

In **d** the result of the right shift gives a value of 4, which is incorrect since $36 \div 2^3$ is not 4; therefore, the number of possible right shifts has been exceeded. You can also see that a 1 has been lost from the original binary number, which is another sign that there have been too many right shifts.

Activity 1.11

1 a Write down the denary value of the following binary number.

0	1	1	0	1	0	0	0

b Shift the binary number three places to the right and comment on your result.

c Write down the denary value of the following binary number.

0	0	0	0	1	1	1	1

d Shift the binary number four places to the left and comment on your result.

2 a Convert 29 and 51 to 8-bit binary numbers.

b Add the two binary numbers in part **a**.

c Shift the result in part **b** three places to the right.

d Convert 75 to an 8-bit binary number.

e Add the two binary numbers from parts **c** and **d**.

f Shift your result from part **e** one place to the left.

1.1.6 Two's complement (binary numbers)

Up until now, we have assumed all binary numbers are positive integers. To allow the possibility of representing negative integers we make use of **two's complement**. In this section we will again assume 8-bit registers are being used. Only one minor change to the binary headings needs to be introduced here:

−128	64	32	16	8	4	2	1

In two's complement the left-most bit is changed to a negative value. For instance, for an 8-bit number, the value 128 is now changed to –128, but all the other headings remain the same. This means the new range of possible numbers is: –128 (10000000) to +127 (01111111).

It is important to realise when applying two's complement to a binary number that the left-most bit always determines the sign of the binary number. A 1-value in the left-most bit indicates a negative number and a 0-value in the left-most bit indicates a positive number (for example, 00110011 represents 51 and 11001111 represents –49).

Writing positive binary numbers in two's complement format

Example 1

The following two examples show how we can write the following positive binary numbers in the two's complement format 19 and 4:

–128	64	32	16	8	4	2	1
0	0	0	1	0	0	1	1
0	0	0	0	0	1	0	0

As you will notice, for positive binary numbers, it is no different to what was done in Section 1.1.2.

Converting positive denary numbers to binary numbers in the two's complement format

If we wish to convert a positive denary number to the two's complement format, we do exactly the same as in Section 1.1.2:

Example 2

Convert **a** 38 **b** 125 to 8-bit binary numbers using the two's complement format.

a Since this number is positive, we must have a zero in the –128 column. It is then a simple case of putting 1-values into their correct positions to make up the value of 38:

–128	64	32	16	8	4	2	1
0	0	1	0	0	1	1	0

b Again, since this is a positive number, we must have a zero in the –128 column. As in part **a**, we then place 1-values in the appropriate columns to make up the value of 125:

–128	64	32	16	8	4	2	1
0	1	1	1	1	1	0	1

Converting positive binary numbers in the two's complement format to positive denary numbers

Example 3

Convert 01101110 in two's complement binary into denary:

–128	64	32	16	8	4	2	1
0	1	1	0	1	1	1	0

As in Section 1.1.2, each time a 1 appears in a column, the column value is added to the total. For example, the binary number (01101110) above has the following denary value: 64 + 32 + 8 + 4 +2 = 110.

Example 4

Convert 00111111 in two's complement binary into denary:

–128	64	32	16	8	4	2	1
0	0	1	1	1	1	1	1

As above, each time a 1 appears in a column, the column value is added to the total. For example, the binary number (00111111) above has the following denary value: 32 + 16 + 8 + 4 +2 + 1 = 63.

Activity 1.12

1 Convert the following positive denary numbers into 8-bit binary numbers in the two's complement format:

a	39	**c**	88	**e**	111	**g**	77	**i**	49
b	66	**d**	102	**f**	125	**h**	20	**j**	56

2 Convert the following binary numbers (written in two's complement format) into positive denary numbers:

	–128	64	32	16	8	4	2	1
a	0	1	0	1	0	1	0	1
b	0	0	1	1	0	0	1	1
c	0	1	0	0	1	1	0	0
d	0	1	1	1	1	1	1	0
e	0	0	0	0	1	1	1	1
f	0	1	1	1	1	1	0	1
g	0	1	0	0	0	0	0	1
h	0	0	0	1	1	1	1	0
i	0	1	1	1	0	0	0	1
j	0	1	1	1	1	0	0	0

Writing negative binary numbers in two's complement format and converting to denary

Example 1

The following three examples show how we can write negative binary numbers in the two's complement format:

–128	64	32	16	8	4	2	1
1	0	0	1	0	0	1	1

By following our normal rules, each time a 1 appears in a column, the column value is added to the total. So, we can see that in denary this is: –128 + 16 + 2 + 1 = –109.

–128	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

Similarly, in denary this number is –128 + 64 + 32 + 4 = –28.

–128	64	32	16	8	4	2	1
1	1	1	1	0	1	0	1

This number is equivalent to –128 + 64 + 32 + 16 + 4 + 1 = –11.

Note that a two's complement number with a 1-value in the -128 column must represent a negative binary number.

Converting negative denary numbers into binary numbers in two's complement format

Consider the number +67 in 8-bit (two's complement) binary format:

–128	64	32	16	8	4	2	1
0	1	0	0	0	0	1	1

Method 1

Now let's consider the number -67. One method of finding the binary equivalent to -67 is to simply put 1s in their correct places:

–128	64	32	16	8	4	2	1
1	0	1	1	1	1	0	1

–128 + 32 + 16 + 8 + 4 + 1 = –67

Method 2

However, looking at the two binary numbers above, there is another possible way to find the binary representation of a negative denary number:

first write the number as a positive binary value – in this case 67:	0 1 0 0 0 0 1 1
we then invert each binary value, which means swap the 1s and 0s around:	1 0 1 1 1 1 0 0
then add 1 to that number:	1
this gives us the binary for –67:	1 0 1 1 1 1 0 1

Example 2

Convert –79 into an 8-bit binary number using two's complement format.

Method 1

As it is a negative number, we need a 1-value in the –128 column.

–79 is the same as –128 + 49

We can make up 49 from 32 + 16 + 1; giving:

–128	64	32	16	8	4	2	1
1	0	1	1	0	0	0	1

Method 2

write 79 in binary:	0 1 0 0 1 1 1 1
invert the binary digits:	1 0 1 1 0 0 0 0
add 1 to the inverted number	1
thus giving –79:	1 0 1 1 0 0 0 1

–128	64	32	16	8	4	2	1
1	0	1	1	0	0	0	1

It is a good idea to practise both methods.

When applying two's complement, it isn't always necessary for a binary number to have 8 bits:

Example 3

The following 4-bit binary number represents denary number 6:

–8	4	2	1
0	1	1	0

Applying two's complement (1 0 0 1 + 1) would give:

–8	4	2	1
1	0	1	0

in other words: –6

Example 4

The following 12-bit binary number represents denary number 1676:

−2048	1024	512	256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	0	0	1	1	0	0

Applying two's complement (1 0 0 1 0 1 1 1 0 0 1 1 + 1) would give:

−2048	1024	512	256	128	64	32	16	8	4	2	1
1	0	0	1	0	1	1	1	0	1	0	0

In other words: −1676

Activity 1.13

Convert the following negative denary numbers into binary numbers using the two's complement format:

a −18 **b** −31 **c** −47 **d** −63 **e** −88 **f** −92 **g** −100 **h** −1 **i** −16 **j** −127

Activity 1.14

Convert the following negative binary numbers (written in two's complement format) into negative denary numbers:

a	1	1	0	0	1	1	0	1
b	1	0	1	1	1	1	1	0
c	1	1	1	0	1	1	1	1
d	1	0	0	0	0	1	1	1
e	1	0	1	0	0	0	0	0
f	1	1	1	1	1	0	0	1
g	1	0	1	0	1	1	1	1
h	1	1	1	1	1	1	1	1
i	1	0	0	0	0	0	0	1
j	1	1	1	1	0	1	1	0

1.2 Text, sound and images

1.2.1 Character sets – ASCII code and Unicode

The **ASCII code** system (American Standard Code for Information Interchange) was set up in 1963 for use in communication systems and computer systems. A newer version of the code was published in 1986. The standard ASCII code **character set** consists of 7-bit codes (0 to 127 in denary or 00 to 7F in